Lab	Type	Practical
Basic programs of C		
1	**A**	1. WAP to calculate area of a Circle ($A = \pi r^2$).
	A	2. WAP to find whether a number is odd or even.
	A	3. WAP to determine whether the entered character is vowel or not.
	A	4. WAP to find the largest among the given three numbers by user.
	A	5. WAP to check for the leap year.
	B	6. WAP to convert seconds into hours, minutes & seconds and print in HH:MM:SS [e.g. 10000 seconds mean 2:46:40 (2 Hours, 46 Minutes, 40 Seconds)].
	C	7. WAP to convert number of days into year, week & days [e.g. 375 days mean 1 year, 1 week and 3 days].
2	**A**	8. WAP to find factorial of a number using loop.
	A	9. WAP to find power of a number using loop.
	A	10. WAP to reverse a number.
	A	11. WAP to find factors of a given number.
	A	12. WAP to check whether a number is prime or not.
	B	13. WAP to print prime numbers between given interval.
	B	14. WAP to find the sum of 1 + (1+2) + (1+2+3) + (1+2+3+4)+ …+(1+2+3+4+….+n).
	C	15. WAP to print Armstrong number from 1 to 1000.
Unit: I – Array		
3	**A**	16. WAP to read and display n numbers using an array.
	A	17. WAP to calculate sum of numbers from m to n.
	A	18. WAP to calculate average of first n numbers.
	A	19. WAP to find position of the smallest number & the largest number from given n numbers.
	B	20. WAP to find whether the array contains a duplicate number or not.

Lab	Type	Practical
	B	21. Read n numbers in an array then read two different numbers, replace 1st number with 2nd number in an array and print its index and final array.
	C	22. WAP to print Pascal triangle.
	C	23. WAP to sort the N names in an alphabetical order.
4	A	24. Write a program to insert a number at a given location in an array.
	A	25. Write a program to delete a number from a given location in an array.
	A	26. WAP to insert a number in an array that is already sorted in an ascending order.
	B	27. WAP to delete a number from an array that is already sorted in an ascending order.
	B	28. WAP to delete duplicate numbers from an array.
	B	29. WAP to merge two unsorted arrays.
	C	30. WAP to replace lowercase characters by uppercase & vice-versa in a user specified string.
5	A	31. Read two 2x2 matrices and perform addition of matrices into third matrix and print it
	A	32. Read two matrices, first 3x2 and second 2x3, perform multiplication operation and store result in third matrix and print it.
	B	33. **Anagram Game** • Allow a user to enter N words and store it in an array. • Generate a random number between 0 to N-1. • Based on the random number generated display the word stored at that index of an array and allow user to enter its anagram. • Check whether the word entered by the user is an anagram of displayed number or not and display an appropriate message. • Given a word A and word B. B is said to be an anagram of A if and only if the characters present in B is same as characters present in A, irrespective of their sequence. For ex: “LISTEN” == “SILENT”

Lab	Type	Practical
Unit: I – Stack		
6	**A**	34. Write a menu driven program to implement following operations on the Stack using an Array • PUSH, POP, DISPLAY • PEEP, CHANGE
	A	35. How stack can be used to recognize strings aca, bcb, abcba, abbcbba? WAP to solve the above problem.
	B	36. WAP to determine if an input character string is of the form a^ib^i where i >= 1 i.e., Number of 'a' should be equal to number of 'b'.
7	**A**	37. **Valid Parenthesis Problem** Chef has a string which contains only the characters '{', '}', '[', ']', '(' and ')'. Now Chef wants to know if the given string is balanced or not. If is balanced then print 1, otherwise print 0. A balanced parenthesis string is defined as follows: • The empty string is balanced • If P is balanced then (P), {P}, [P] is also balanced • if P and Q are balanced PQ is also balanced • "([])", "({})[()]" are balanced parenthesis strings • "([{]})", "())" are not balanced. **Input Format:** The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows. The first and only line of each test case contains a single string **Output Format:** For each test case, print a single line containing the answer. **Sample Example:** Input: 4 () ([]) ([{}()])[{}] [{{}] Output: 1 0 1 0

Lab	Type	Practical
	A	38. **Removing starts from a string Problem** You are given a string s, which contains stars *. In one operation, you can: Choose a star in s. Remove the closest non-star character to its left, as well as remove the star itself. Return the string after all stars have been removed. **Note :** • The input will be generated such that the operation is always possible. • It can be shown that the resulting string will always be unique. **Sample Example-1:** **Input:** s = "leet**cod*e". **Output:** "lecoe" **Sample Example-2:** **Input:** s = "erase*****" **Output:** ""
	B	39. **Merge Intervals Problem** Given a set of time intervals in any order, our task is to merge all overlapping intervals into one and output the result which should have only mutually exclusive intervals. **Sample Example-1:** **Input:** Intervals = {{1,3},{2,4},{6,8},{9,10}} **Output:** {{1, 4}, {6, 8}, {9, 10}} **Explanation:** Given intervals: [1,3],[2,4],[6,8],[9,10], we have only two overlapping intervals here,[1,3] and [2,4]. Therefore we will merge these two and return [1,4],[6,8], [9,10] **Sample Example-2:** **Input:** Intervals = {{6,8},{1,9},{2,4},{4,7}} **Output:** {{1, 9}}

<table>
<tr><th>Lab</th><th>Type</th><th>Practical</th></tr>
<tr><td></td><td>C</td><td>40. Vowel Anxiety Problem
Utkarsh has recently started taking English-language classes to improve his reading and writing skills. However, he is still struggling to learn English. His teacher gave him the following problem to improve his vowel-identification skills:

There is a string S of length N consisting of lowercase English letters only.
Utkarsh has to start from the first letter of the string.
Each time he encounters a vowel; he has to reverse the entire substring that came before the vowel.
Utkarsh needs help verifying his answer. Can you print the final string after performing all the operations for him?

Input Format:
• First line will contain T, number of test cases. Then T test cases follow.
• The first line of each test case contains N, the length of the string.
• The second line contains S, the string itself.

Output Format:
For each test case, output in a single line the final string after traversing S from left to right and performing the necessary reversals.

Sample Example:
<table><tr><td>Input:

2
10
abcdefghij
7
bcadage</td></tr></table><table><tr><td>Output:

hgfeabcdij
gacbade</td></tr></table></td></tr>
<tr><td>8</td><td>A</td><td>41. WAP to convert infix notation to postfix notation using stack.</td></tr>
<tr><td></td><td>B</td><td>42. WAP to convert infix notation to prefix notation using stack.</td></tr>
<tr><td>9</td><td>A</td><td>43. WAP for evaluation of postfix Expression using Stack.</td></tr>
<tr><td></td><td>B</td><td>44. WAP for evaluation of prefix Expression using Stack.</td></tr>
</table>

Lab	Type	Practical
	C	45. You have an array A of integers of size N, an array B (initially empty) and a stack S (initially empty). You are allowed to do the following operations: • Take the first element of array A and push it into S and remove it from A. • Take the top element from stack S, append it to the end of array B and remove it from S. You have to tell if it possible to move all the elements of array A to array B using the above operations such that finally the array B is sorted in ascending order. **Input Format :** • First line will contain T, number of testcases. Then the testcases follow. • First line of each testcase contains a single integer N. • Second line of each testcase contains N distinct integers : A_1,A_2...A_N. **Output Format :** For each testcase, if it possible to move all the elements of array A to array B using the above operations such that finally, the array B is sorted in ascending order, print "YES" (without quotes), else print "NO" (without quotes). **Sample Example:** Input: 2 4 1 2 4 3 4 1 3 4 2 Output: YES NO
10	**A** **A** **A** **A** **A** **B** **B**	46. Write a recursive function to generate the nth Fibonacci number. 47. Write a recursive function to find the sum of digits of a given number. 48. Write a recursive function to reverse a given number. 49. Write a recursive function to find the largest element in a given array. 50. Write a recursive function to calculate the sum of all elements in an array. 51. Write a recursive function to determine whether a given string is a palindrome. 52. Write a recursive function to convert a decimal number into its binary equivalent.

C

53. **Tower of Hanoi Using Stack**

The Tower of Hanoi is a puzzle consisting of three rods and n disks of different sizes. Initially, all disks are stacked on the Source rod in decreasing order of size, with the largest disk at the bottom and the smallest disk at the top.

Your task is to simulate the movement of disks from the Source rod to the Destination rod using an iterative approach based on the Stack data structure.

The following rules must be followed:

- Only one disk can be moved at a time.
- A disk can only be moved from the top of one rod to the top of another rod.
- A larger disk cannot be placed on top of a smaller disk.

Print the sequence of moves required to transfer all disks from the Source rod to the Destination rod.

Input Format
A single integer n, representing the number of disks.

Constraints
$1 \le n \le 10$

Output Format
Print each move on a new line in the following format:
Move disk X from rod Y to rod Z

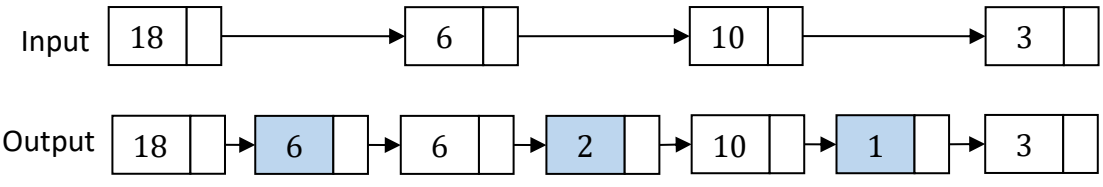
where:

- X is the disk number.
- Y is the source rod.
- Z is the destination rod.

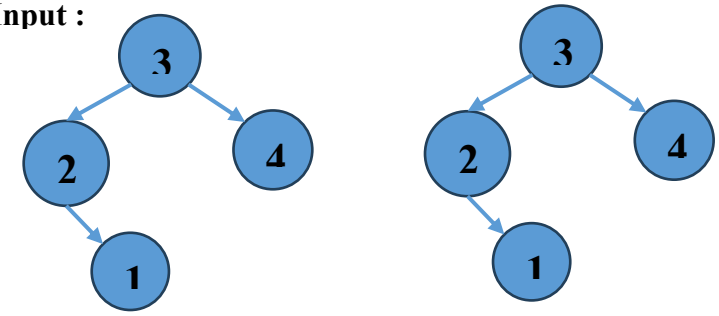
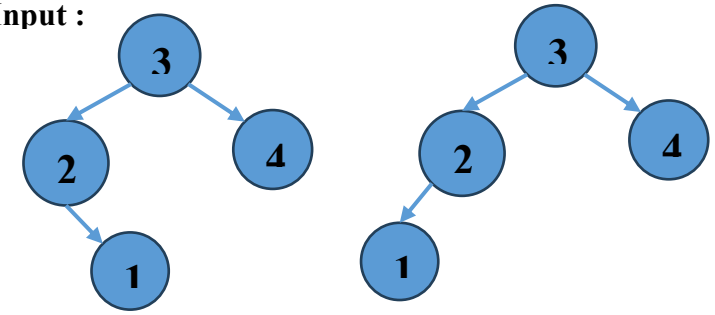
Sample Input 1:
2
Sample Output 1:
Move disk 1 from rod S to rod A
Move disk 2 from rod S to rod D
Move disk 1 from rod A to rod D

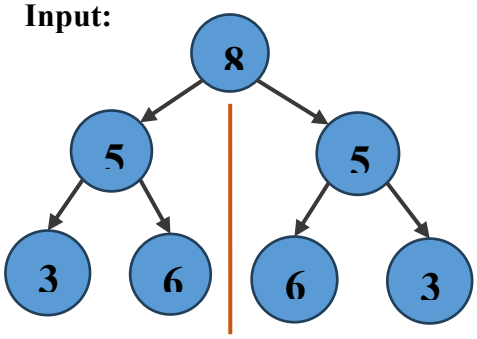
Sample Input 2:
3
Sample Output 2:
Move disk 1 from rod S to rod D
Move disk 2 from rod S to rod A
Move disk 1 from rod D to rod A

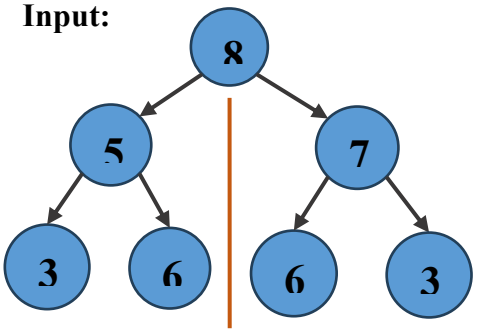
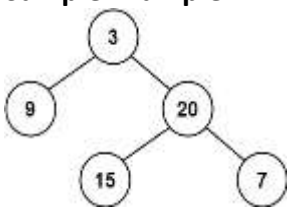
Lab	Type	Practical
		Move disk 3 from rod S to rod D Move disk 1 from rod A to rod S Move disk 2 from rod A to rod D Move disk 1 from rod S to rod D
Unit: II – Queue		
11	**A**	54. Write a menu driven program to implement following operations on the Queue using an Array • ENQUEUE • DEQUEUE • DISPLAY
	A	55. Write a menu driven program to implement following operations on a circular queue using an Array • Insert • Delete • Display all elements of the queue
12	**A**	56. Write a menu driven program to implement following operations on the Doubled Ended Queue using an Array • Insert at front end, Insert at rear end • Delete from front end, Delete from rear end • Display all elements of the queue
	A	57. WAP to implement Priority Queue. (Ascending / Descending)
Unit: II – Linked List		
13	**A** **A** **A** **A** **B** **B** **C**	58. WAP to allocate and de-allocate memory for int, char and float variable at runtime. 59. WAP to get and print the array elements using Pointer. 60. WAP to calculate the sum of n numbers using Pointer. 61. WAP to find the largest element in the array using Pointer. 62. WAP to sort the array elements using Pointer. 63. WAP to check whether the string is Palindrome or not using Pointer. 64. WAP to define a C structure named Student (roll_no, name, branch and batch_no) and also to access the structure members using Pointer.

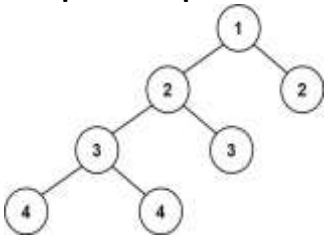
Lab	Type	Practical
14	**A**	65. Write a menu driven program to implement following operations on the singly linked list. • Insert a node at the front of the linked list. • Insert a node at the end of the linked list. • Insert a node by preserving the order of the data of the linked list. (Ordered insertion) • Display all nodes. • Delete the first node of the linked list. • Delete the last node of the linked list. • Delete a node from specified position of the list. • Count the no. of nodes in the linked list.
15	**A**	66. WAP to copy a linked list. 67. WAP to reverse a linked list. 68. WAP to sort elements of a linked list.
16	**A**	69. WAP to remove the duplicates nodes from given sorted Linked List. Input: 1 → 1 → 6 → 13 → 13 → 13 → 27 → 27 Output: 1 → 6 → 13 → 27
	B	70. WAP to check whether 2 singly linked lists are same or not.
	C	71. WAP to remove duplicate elements from a singly linked list.
17	**A**	72. WAP to implement Stack using singly linked list.
	A	73. WAP to implement Simple Queue, Circular Queue & Priority Queue using singly linked list.
18	**A**	74. WAP to swap K^{th} node from beginning with K^{th} node from end in a singly linked list.
	B	75. WAP to perform given operation in the linked list. There exist a Linked List. Add a node that contains the GCD of those two nodes between every pair adjacent node of Linked List. 

Lab	Type	Practical
	C	76. WAP to swap two consecutive nodes in the linked list. Don't change the values of nodes, implement by changing the link of the nodes. • Input: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 • Output: 2 → 1 → 4 → 3 → 6 → 5 → 8 → 7
19	A	77. Write a menu driven program to implement following operations on the circular linked list. • Insert a node at the front of the list. • Insert a node at the end of the list. • Delete the first node of the list. • Delete the last node of the list. • Delete a node from specified position of the list. • Display all nodes.
	B	78. WAP to split a circular linked list into two halves.
	C	79. WAP to perform addition of two polynomial equations using appropriate data structure.
20	A	80. Write a menu driven program to implement following operations on the doubly linked list. • Insert a node at the front of the linked list. • Delete a node from specified position. • Insert a node at the end of the linked list. • Display all nodes.
	B	81. WAP to delete alternate nodes of a doubly linked list.
	C	82. **Music Player Simulation** WAP to simulate music player application using suitable data structure. There is no estimation about number of music files to be managed by the music player. Your program should support all the basic music player operations to play and manage the playlist.
Unit: III – Nonlinear Data Structures: Tree and Graph		

Lab	Type	Practical
21	A	83. Write a menu driven program to implement Binary Search Tree (BST) & perform following operations: • Insert a node • Delete a node • Search a node • Preorder Traversal • Postorder Traversal • Inorder Traversal
	B	84. WAP to check whether the given two trees are same or not. **Input :**  **Output:** Given trees are same **Input :**  **Output:** Given trees are not same

<table>
<tr><th>Lab</th><th>Type</th><th>Practical</th></tr>
<tr><td rowspan="4">22</td><td>A</td><td>85. WAP to check whether the given tree is symmetric or not.
Input:
 
Output: Given tree is symmetric

Input:
 
Output: Given tree is not symmetric</td></tr>
<tr><td>B</td><td>86. Given a binary tree, determine if it is height-balanced (A height-balanced binary tree is a binary tree in which the depth of the two subtrees of every node never differs by more than one).
Sample Example-1:
 
Input: root = [3,9,20,null,null,15,7]
Output: TRUE

Sample Example-2:
 
Input: root = [1,2,2,3,3,null,null,4,4]
Output: FALSE</td></tr>
<tr><td>C</td><td>87. WAP to implement Red-Black Tree.</td></tr>
<tr><td>C</td><td>88. WAP to implement 2-3 Tree.</td></tr>
</table>

Lab	Type	Practical
23	A	89. WAP to find the smallest and largest elements in the Binary Search Tree.
	B	90. WAP to implement phone book dictionary using Binary Search Tree which provides following operations: • Add new entry in phone book • Remove entry from phone book • Search phone number • List all entries in ascending order of name • List all entries in descending order of name
	C	91. WAP to construct a binary tree from given Inorder and Preorder traversal sequence.
24	A	92. WAP to create a graph & implement the adjacency list representation of the graph. Apply DFS and BFS on the given graph to traverse the created Graph.
	C	93. You are given an undirected graph with N nodes (numbered 1 through N). For each valid i, the i-th node has a weight Wi. Also, for each pair of nodes i and j, there is an edge connecting nodes if $j - i \neq W_j - W_i$. Find the number of connected components in this graph. **Input Format:** • The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows. • The first line of each test case contains a single integer N. • The line contains N space-separated integers W_1, W_2, ..., W_N. **Output Format :** For each test case, print a single line containing one integer --- the number of components in the graph. **Sample Example:** Input: 2 2 1 2 2 2 1 Output: 2 1

Lab	Type	Practical
Unit: IV – Hashing		
25	**A**	94. In an array of 20 elements, arrange 15 different values, which are generated randomly between 1,00,000 to 9,99,999. Use hash function to generate key and linear probing to avoid collision. H(x) = (x mod 18) + 2. Write a program to input and display the final values of array.
	A	95. Implement Hash set.
	B	96. Implement a Hash Map/Dictionary (key, value) pair using Hash-table.
Unit: V – Sorting and Searching		
26	**A**	97. WAP to implement a Linear Search using Array.
	A	98. WAP to implement a Binary Search using Array. (Iterative & recursive)
27	**A**	99. WAP to implement Bubble using Array.
	A	100. WAP to implement Selection Sort using Array.
28	**A**	101. WAP to implement Insertion Sort using Array.
	A	102. WAP to implement Merge Sort using Array.
29	**A**	103. WAP to implement Quick Sort using Array.
30	**A**	104. Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue. We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively. **Sample Example-1:** **Input:** nums = [2,0,2,1,1,0] **Output:** [0,0,1,1,2,2] **Sample Example-2:** **Input:** nums = [2,0,1] **Output:** [0,1,2]

Lab	Type	Practical
	B	105. **Sliding Window Problem** You are given an array of positive integers and an integer K. Find the length of the longest subarray such that the sum of the subarray is less than or equal to K. **Input:** • An array arr[] of size N • An integer K **Output:** • Length of the longest subarray with sum ≤ K **Example:** **Input:** arr = [1, 2, 1, 0, 1, 1, 0] K = 4 **Output:** 5 **Explanation:** Subarrays with sum ≤ 4: • [1, 2, 1] → sum = 4 → length = 3 • [2, 1, 0, 1] → sum = 4 → length = 4 • [1, 2, 1, 0, 1] → sum = 5 → too big • [1, 0, 1, 1, 0] → sum = 3 → length = 5 (maximum) **Idea (Sliding Window):** • Use two pointers start and end to maintain a window. • Add elements from the right (end) into the current sum. • If the sum becomes > K, move start forward until sum ≤ K. • Keep track of the maximum window size where sum ≤ K.